

# Practice 2 – Stretch Robot Kinematics

Robotics and Cyber-Physical Systems  
School of Sciences and Engineering  
Tecnológico de Monterrey

## Introduction

In this practice, you will control the Stretch robot using two complementary interfaces. First, you will use the robot's standard Python API to command exact joint positions. You will then use the `stretch_tools` package to send normalized velocity commands and combine autonomous motion with manual input.

The same scripts can run in the MuJoCo simulator and on the physical robot. The simulator package implements the most important parts of the physical Stretch API, allowing the backend to be selected with a simple import fallback.

The practice has three parts:

1. Move the robot through a fixed grasping sequence using position commands.
2. Drive one joint with a sine-wave velocity while reading its position.
3. Add full keyboard and gamepad override to the sine-wave behavior.

## Before You Begin

Update the repository and synchronize its dependencies before starting. Back up any uncommitted work before using commands that overwrite local changes.

```
1 git pull
2 uv sync
```

Always enable collision management before moving the physical robot:

```
1 stretch.enable_collision_mgmt()
```

Make sure the robot and its surroundings have enough clearance before running any program. Keep the emergency stop accessible when working with the physical robot.

## 1 Part 1: Position Commands with the Robot API

The standard Stretch API begins with a `Robot` object. Use the following import pattern so the same file selects the physical API when it is installed and falls back to the simulator otherwise:

```
1 try:
2     import stretch_body.robot as robot
3 except ImportError:
4     import stretch_mujoco_api.robot as robot
```

Create, start, and safely stop the robot as follows:

```

1 if __name__ == "__main__":
2     stretch = robot.Robot()
3     stretch.startup()
4     stretch.enable_collision_mgmt()
5
6     try:
7         # Robot commands go here.
8         pass
9     finally:
10        stretch.stop()

```

The `finally` block runs even if the program encounters an error or the user presses Ctrl+C. It ensures that the robot connection and simulator process are closed cleanly.

## Accessing the joints

The lift and arm are available directly from the robot:

```

1 lift = stretch.lift
2 arm = stretch.arm

```

Head and end-of-arm joints are retrieved by name:

```

1 head_pan = stretch.head.get_joint("head_pan")
2 head_tilt = stretch.head.get_joint("head_tilt")
3
4 wrist_pitch = stretch.end_of_arm.get_joint("wrist_pitch")
5 wrist_roll = stretch.end_of_arm.get_joint("wrist_roll")
6 wrist_yaw = stretch.end_of_arm.get_joint("wrist_yaw")
7 gripper = stretch.end_of_arm.get_joint("stretch_gripper")

```

## Movement commands

The principal joint methods are:

| Method                              | Behavior                                    |
|-------------------------------------|---------------------------------------------|
| <code>move_to(position)</code>      | Move to an absolute position                |
| <code>move_by(distance)</code>      | Move relative to the current position       |
| <code>set_velocity(velocity)</code> | Move continuously at the requested velocity |

Lift and arm positions are measured in meters. Head and wrist positions are measured in radians. The gripper uses percentage-like commands, normally in the range  $[-100, 100]$ , where positive values open the gripper and negative values close it farther.

A position command follows this pattern:

```

1 stretch.lift.move_to(0.5)
2 stretch.push_command()
3 stretch.wait_command()

```

`push_command()` sends queued commands to the robot. `wait_command()` blocks until the commanded motion is complete. Always push each movement before waiting for it. Waiting after every step is especially important when the order of operations prevents collisions.

Joint state is available through the `status` dictionary:

```

1 position = stretch.lift.status["pos"]
2 velocity = stretch.lift.status["vel"]
3 print(position, velocity)

```

The complete robot state can be retrieved with `stretch.get_status()`.

### Activity: Create `p2_1_grab.py`

Create a new Python file named `p2_1_grab.py` from scratch. Do not copy a completed solution. Include the cross-environment import, robot startup, collision management, and a `try/finally` cleanup block.

Inside the `try` block, command the following sequence in this exact order:

1. Move the lift to 0.41 m.
2. Open the gripper to 50.
3. Extend the arm to 0.14 m.
4. Close the gripper toward 0.5.
5. Raise the lift to 0.60 m.

Call `stretch.push_command()` and `stretch.wait_command()` after every movement. The gripper may stop before reaching 0.5 because the object prevents its fingers from fully closing. This is normal and represents a successful grasp.

Run the program with:

```
1 uv run p2_1_grab.py
```

**Expected result:** The robot lowers to the object, opens its gripper, extends its arm, grasps the object, and lifts it. The ordering should prevent the arm and gripper from colliding with the environment.

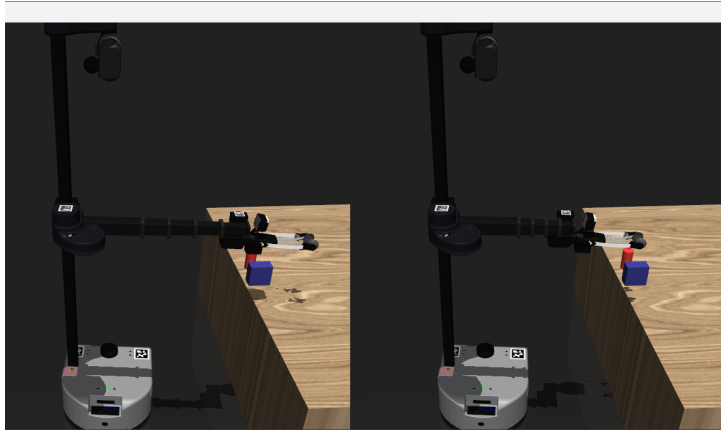


Figure 1: The arm extending as part of the grasping sequence.

## 2 Part 2: Sinusoidal Velocity Control and Joint State

Position commands are useful for discrete motions. Continuous behaviors are often easier to express as normalized joint velocities. The `NormVelController` class from `stretch_tools` accepts sparse command dictionaries with values in  $[-1, 1]$ :

```
1 from stretch_tools import NormVelController
2
3 controller = NormVelController(stretch)
4 controller.set_command({"head_pan_counterclockwise": 0.5})
```

A value of 1 requests the configured maximum speed in the positive named direction. A value of  $-1$  requests maximum speed in the opposite direction. A value of 0 stops and holds that joint.

Command dictionaries are sparse. If a key is omitted, the controller does not send any command to that joint. This allows omitted joints to continue following commands issued through the standard robot API.

The available normalized command keys are:

| Key                                      | Positive direction      |
|------------------------------------------|-------------------------|
| <code>base_forward</code>                | Base translates forward |
| <code>base_counterclockwise</code>       | Base rotates left       |
| <code>lift_up</code>                     | Lift moves upward       |
| <code>arm_out</code>                     | Arm extends             |
| <code>wrist_roll_counterclockwise</code> | Wrist rolls left        |
| <code>wrist_pitch_up</code>              | Wrist pitches upward    |
| <code>wrist_yaw_counterclockwise</code>  | Wrist yaws left         |
| <code>head_pan_counterclockwise</code>   | Head pans left          |
| <code>head_tilt_up</code>                | Head tilts upward       |
| <code>gripper_open</code>                | Gripper opens           |

### Activity: Create `p2_2_sine.py`

Create a new script named `p2_2_sine.py`. Set up the robot using the same import and lifecycle pattern from Part 1, then instantiate a `NormVelController`.

Make the head pan continuously using the normalized velocity:

$$v(t) = \sin(\omega t), \quad \omega = 2.0$$

Use `time.monotonic()` to measure elapsed time:

```

1 started = time.monotonic()
2
3 while True:
4     elapsed = time.monotonic() - started
5     velocity = math.sin(2.0 * elapsed)

```

On every iteration:

1. Calculate the sinusoidal velocity.
2. Send it using the `head_pan_counterclockwise` command key.
3. Read the head-pan position from `stretch.head.get_joint("head_pan").status["pos"]`.
4. Print the velocity and position.
5. Sleep for 0.1 seconds before the next update.

Your loop should contain the following core operations:

```

1 command = {"head_pan_counterclockwise": velocity}
2 controller.set_command(command)
3
4 position = head_pan.status["pos"]
5 print(f"velocity={velocity:.3f}, position={position:.3f}")
6 time.sleep(0.1)

```

Run the program with:

```
1 uv run p2_2_sine.py
```

**Expected result:** The head pans smoothly left and right. The terminal prints a sinusoidal normalized velocity and the corresponding head angle in radians.

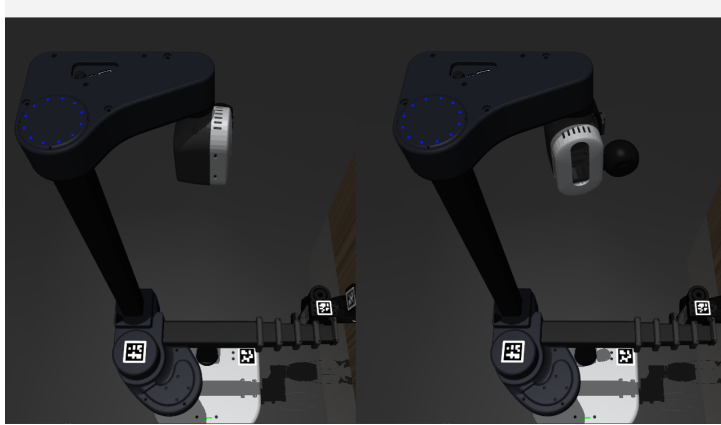


Figure 2: Head-pan motion produced by a sinusoidal velocity command.

### 3 Part 3: Manual Override

Autonomous and manual commands can be combined through `TeleopProvider`. It reads the configured keyboard and gamepad inputs and uses proportional blending to override an algorithmic command only where the operator provides input.

Import and create it as follows:

```
1 from stretch_tools import NormVelController, TeleopProvider
2
3 controller = NormVelController(stretch)
4 teleop = TeleopProvider(robot=stretch)
```

Given an autonomous command dictionary, obtain the final command with:

```
1 command = teleop.get_manual_override(command_autonomous)
2 controller.set_command(command)
```

In the default autonomous mode, idle teleop inputs leave the autonomous command unchanged. Moving a stick or holding a mapped key proportionally overrides the affected joint. Full input takes complete control of that joint.

Press the Y face button or the y key to toggle manual mode. In manual mode, the autonomous dictionary is ignored and only teleop commands are sent. Press it again to restore autonomous motion with proportional override.

Useful keyboard controls include:

| Keys | Control                                               |
|------|-------------------------------------------------------|
| W/S  | Base forward/backward                                 |
| A/D  | Base rotation                                         |
| Z/X  | Lift                                                  |
| V/C  | Arm                                                   |
| M/N  | Gripper                                               |
| J/L  | Wrist yaw                                             |
| O/U  | Wrist roll                                            |
| I/K  | Wrist pitch                                           |
| H    | Toggle D-pad control between wrist and head           |
| F    | Hold for fast base mode when the lift is below 0.34 m |
| P    | Hold for precision mode                               |
| Y    | Toggle autonomous/manual mode                         |

The left trigger proportionally slows all joints, reaching 10% speed when fully pressed. The right trigger proportionally increases base speed from the default safe limit to full speed, but only while the lift is below 0.34 m.

### Activity: Create p2\_3\_manual\_override.py

Copy your completed Part 2 program:

```
1 cp p2_2_sine.py p2_3_manual_override.py
```

Then:

1. Import `TeleopProvider` from `stretch_tools`.
2. Create a provider with `TeleopProvider(robot=stretch)`.
3. Keep calculating the same sinusoidal autonomous command.
4. Pass that dictionary to `teleop.get_manual_override()`.
5. Send the returned dictionary through `controller.set_command()`.
6. Continue printing head-pan velocity and position.

The central loop should now follow this pattern:

```
1 command_autonomous = {
2     "head_pan_counterclockwise": velocity,
3 }
4 command = teleop.get_manual_override(command_autonomous)
5 controller.set_command(command)
```

Run the program with:

```
1 uv run p2_3_manual_override.py
```

### Expected result:

- With no input, the head continues following the sine wave.
- Manual input can proportionally override any mapped joint.
- In manual mode, the sinusoidal command is disabled.
- Returning to autonomous mode resumes the sine-wave behavior.

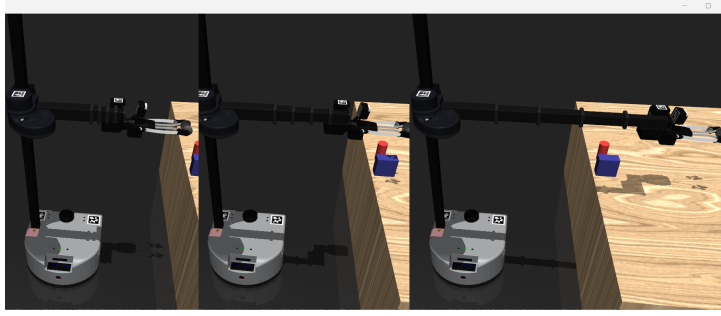


Figure 3: An example of manual input overriding an autonomous joint command.

## Deliverable

Submit the three Python files and a concise video demonstrating:

1. The ordered grasp-and-lift sequence from `p2_1_grab.py`.
2. Sinusoidal head motion and live position output from `p2_2_sine.py`.
3. Autonomous sine-wave motion, proportional manual override, manual-mode toggling, and return to autonomous mode in `p2_3_manual_override.py`.

The video should clearly show that each behavior works. Keep it concise; between one and two minutes is sufficient.